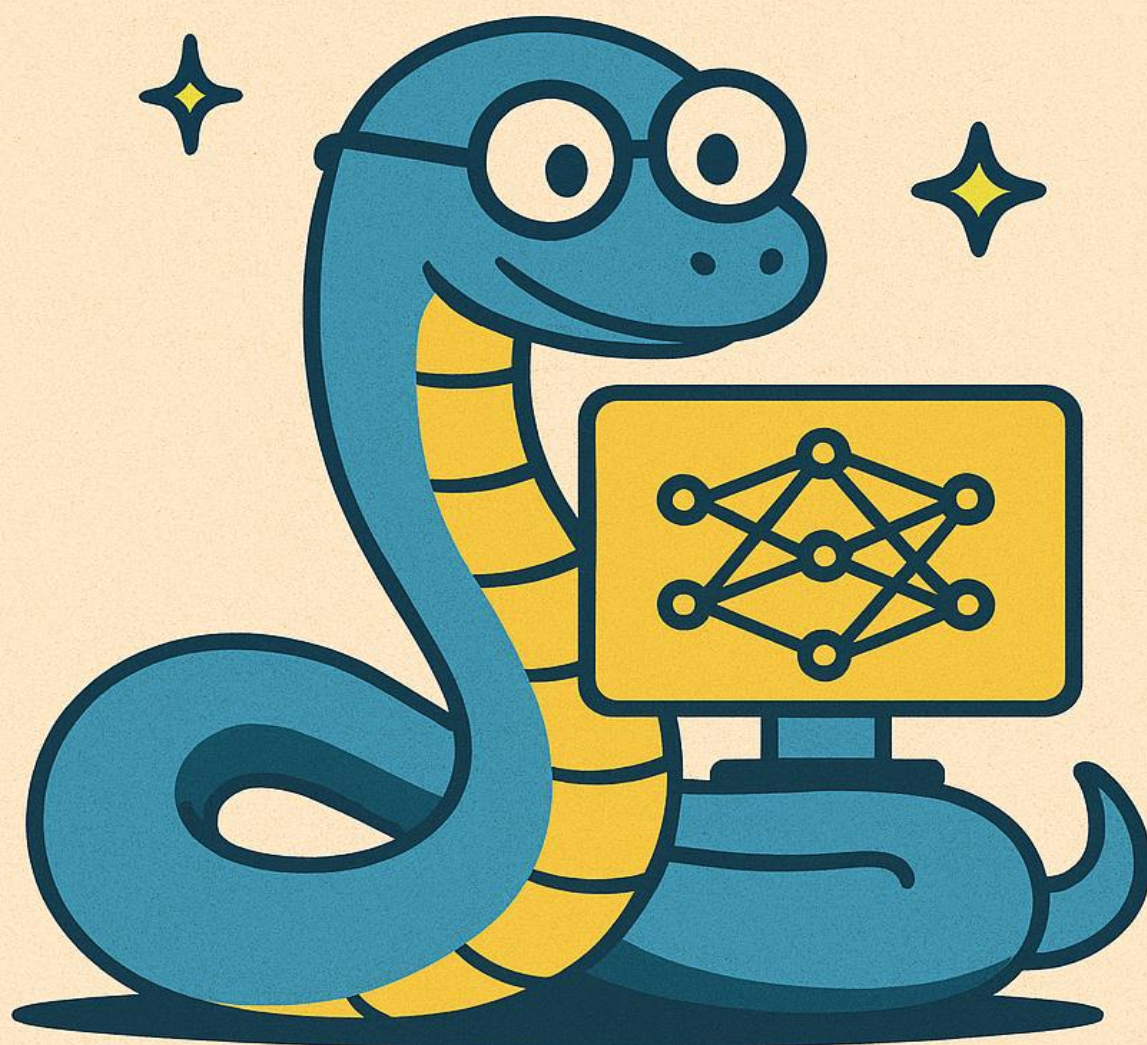


PYTHON SAGA

O PODER DO MACHINE
LEARNING



FRANCIELE FEREREIRA

Introdução

Machine Learning deixou de ser coisa de laboratório e passou a fazer parte do nosso dia a dia — desde a sugestão de filmes que você recebe em plataformas de streaming até o filtro de spam do seu e-mail. E a melhor parte? Você pode começar a trabalhar com isso agora mesmo, usando Python.

Python se tornou a principal linguagem para inteligência artificial e machine learning por um motivo simples: é fácil de aprender, possui uma comunidade enorme e conta com bibliotecas poderosas que reduzem o tempo entre a ideia e a execução.

Neste eBook, você vai aprender como aplicar Machine Learning com Python de forma prática e objetiva. Nada de teoria pesada ou exemplos artificiais — aqui você vai encontrar códigos reais para problemas reais.

01

COMEÇANDO COM OS DADOS

LIMPANDO OS DADOS

Principais bibliotecas do Python:

- **NumPy** → trabalha com matrizes e cálculos numéricos.
- **Pandas** → manipulação de dados.
- **Matplotlib / Seaborn** → visualização.
- **Scikit-learn** → algoritmos de ML prontos.

Antes de treinar qualquer modelo, precisamos analisar e limpar os dados. Exemplo: prever o preço de casas.

```
python

import pandas as pd

# Carregando os dados
dados = pd.read_csv("casas.csv")

# Visualizando as 5 primeiras linhas
print(dados.head())

# Limpando valores ausentes
dados = dados.dropna()

# Selecionando colunas de interesse
X = dados[['quartos', 'banheiros', 'tamanho_m2']]
y = dados['preco']
```

02

TREINANDO UM MODELO DE CLASSIFICAÇÃO

O QUE É CLASSIFICAÇÃO

Classificação é usada quando queremos prever categorias. Exemplo: prever se um tumor é maligno ou benigno com base em características.

Vamos usar o dataset de câncer de mama do scikit-learn.

```
Python

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

# Carrega os dados
dados = load_breast_cancer()
X = dados.data
y = dados.target

# Divide em treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Cria e treina o modelo
modelo = RandomForestClassifier()
modelo.fit(X_train, y_train)

# Faz previsão em 5 casos
pred = modelo.predict(X_test[:5])
print("Previsões:", pred)
```

O que o modelo faz: Aprende com os dados e classifica novos exemplos como "câncer maligno" ou "benigno".

03

**PREVENDO
VALORES COM
REGRESSÃO
LINEAR**

O QUE É REGRESSÃO

É usada quando queremos prever um número contínuo, como preço de casa, salário ou temperatura.

Vamos prever o preço de casas com o dataset de Boston (já carregado no sklearn).

```
Python

from sklearn.datasets import load_boston
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# Carrega os dados
boston = load_boston()
X = boston.data
y = boston.target

# Divide entre treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

# Cria e treina o modelo
modelo = LinearRegression()
modelo.fit(X_train, y_train)

# Previsão
previsoes = modelo.predict(X_test[:5])
print("Preços previstos:", previsoes)
```


04

AGRUPANDO DADOS COM K- MEANS

O QUE É AGRUPAMENTO COM K-MEANS

O algoritmo visa dividir dados em k grupos distintos, onde cada ponto de dados pertence ao grupo com a média mais próxima. O algoritmo itera para refinar essas médias até que os grupos se estabilizem. Vamos agrupar clientes com base em renda e idade.

Python

```
import pandas as pd
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Dados simulados de clientes
data = {
    'idade': [25, 34, 22, 45, 52, 23, 40, 60],
    'renda_mil': [3, 5, 2, 8, 10, 1.5, 7, 12]
}
df = pd.DataFrame(data)

# Aplicar KMeans
kmeans = KMeans(n_clusters=2, random_state=0)
df['grupo'] = kmeans.fit_predict(df)

# Visualizar grupos
plt.scatter(df['idade'], df['renda_mil'], c=df['grupo'], cmap='viridis')
plt.xlabel('Idade')
plt.ylabel('Renda (mil R$)')
plt.title('Agrupamento de Clientes')
plt.show()
```

05

AVALIANDO O MODELO COM MÉTRICAS SIMPLES

AVALIAÇÃO POR CLASSIFICAÇÃO E REGRESSÃO

A avaliação de modelos é crucial para determinar seu desempenho. As métricas variam dependendo se você está lidando com problemas de classificação ou regressão.

Classificação:

Acurácia: Proporção de previsões corretas.

Precisão: Proporção de previsões positivas corretas entre todas as previsões positivas.

Recall (Sensibilidade): Proporção de casos positivos reais que foram corretamente identificados.

F1-Score: Média harmônica de precisão e recall.

AUC-ROC: Área sob a curva ROC, que avalia a capacidade do modelo de distinguir entre classes.

python

```
from sklearn.metrics import classification_report  
  
print(classification_report(y_test, y_pred))
```

Regressão:

Erro Médio Absoluto (MAE): Média da diferença absoluta entre os valores previstos e reais.

Erro Quadrático Médio (MSE): Média do quadrado da diferença entre os valores previstos e reais.

Raiz do Erro Quadrático Médio (RMSE): Raiz quadrada do MSE, mais interpretável que o MSE.

R-quadrado (R^2): Proporção da variância na variável dependente que é previsível a partir das variáveis independentes.

```
python

from sklearn.metrics import mean_squared_error, r2_score

y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"MSE: {mse:.2f}")
print(f"R²: {r2:.2f}")
```

06

Redes Neurais com Python: Do Zero ao Prático

O QUE É UMA REDE NEURAL

Uma **rede neural** é um modelo inspirado no cérebro humano. Ela aprende **padrões complexos** nos dados e é usada em reconhecimento de voz, imagem e texto.

- Em resumo:
- Entrada → Dados (ex: pixels de uma imagem)
- Camadas ocultas → Onde o modelo aprende
- Saída → Resultado (ex: “é um gato” ou “não é um gato”)

Bibliotecas Necessárias:

- Para trabalhar com redes neurais, usaremos:
- **TensorFlow** (ou **Keras**) → construção e treino dos modelos.
- **NumPy** → manipulação de dados.

Classificando Dígitos Escritos à Mão vamos usar o famoso dataset MNIST, com imagens de números de 0 a 9.

python

```
pip install tensorflow numpy

import tensorflow as tf
from tensorflow.keras import layers, models

# Carregando o dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Normalizando os dados (0 a 1)
x_train, x_test = x_train / 255.0, x_test / 255.0

# Criando o modelo
modelo = models.Sequential([
    layers.Flatten(input_shape=(28, 28)), # transforma 28x28 em um vetor
    layers.Dense(128, activation='relu'), # camada oculta
    layers.Dropout(0.2), # evita overfitting
    layers.Dense(10, activation='softmax') # saída (10 classes)
])

# Compilando o modelo
modelo.compile(optimizer='adam',
               loss='sparse_categorical_crossentropy',
               metrics=['accuracy'])

# Treinando
modelo.fit(x_train, y_train, epochs=5)

# Avaliando
modelo.evaluate(x_test, y_test)
```

CLASSIFICAÇÃO DE TEXTO COM REDES NEURAIS

Agora, um exemplo mais prático do dia a dia: classificar avaliações de filmes como positivas ou negativas.

```
python

from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Carregando dados do IMDB
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=10000)

# Padronizando o tamanho dos textos
x_train = pad_sequences(x_train, maxlen=200)
x_test = pad_sequences(x_test, maxlen=200)

# Criando o modelo
modelo = models.Sequential([
    layers.Embedding(10000, 16, input_length=200),
    layers.GlobalAveragePooling1D(),
    layers.Dense(16, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

# Compilando e treinando
modelo.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
modelo.fit(x_train, y_train, epochs=5, batch_size=512, validation_split=0.2)
```

Esse modelo aprende a identificar sentimento em texto — útil para análises de redes sociais, feedbacks, etc.

AGRADECIMENTOS

OBRIGADO POR LER ATÉ AQUI

Esse ebook foi gerado por IA e diagramado por um humano.

Esse conteúdo foi gerado com fins didáticos de construção, não foi realizada uma validação cuidadosa humana no conteúdo e pode conter erros gerados pela IA.

